GrabCut in OpenCV

Complete Step-by-Step Working — Explained in Very Simple Words

Goal: GrabCut separates an object (foreground) from its background. You give OpenCV a rectangle around the object, and GrabCut learns from the image and repeatedly improves the separation.

1. Start with a Very Small Image

Imagine this 5 × 5 image:

B	B	B	B	B
B	B	O	O	B
B	O	O	O	B
B	B	O	O	B
B	B	B	B	B

B = blue background
O = object

Visually:

■ ■ ■ ■ ■
■ ■ ■ ■ ■
■ ■ ■ ■ ■
■ ■ ■ ■ ■
■ ■ ■ ■ ■

Goal: Background ■ Object ■

2. Give GrabCut a Rectangle

Suppose we tell OpenCV: `rect = (1, 1, 3, 3)`.

This means: **"The object is somewhere inside this rectangle."**

Important: GrabCut does not initially know which pixels inside the rectangle are object. Outside the rectangle is treated as definite background; inside the rectangle is uncertain.

3. GrabCut Looks at the Colors

Imagine the background is blue and the object is red:

```
Background: Blue ≈ (20, 30, 200)
Object: Red ≈ (200, 30, 30)
```

GrabCut notices that blue pixels are mostly associated with the background and red pixels are mostly associated with the object. It therefore starts building two models:

BACKGROUND MODEL → learns background colors
FOREGROUND MODEL → learns object colors

Key idea: GrabCut learns what the foreground and background look like from the image itself.

4. It Examines Each Pixel

For each uncertain pixel, GrabCut asks: **“Does this pixel look more like my foreground model or my background model?”**

Example 1:

Pixel = (198, 32, 35)
This is close to the learned red/object colors → **FOREGROUND**.

Example 2:

Pixel = (21, 29, 198)
This is close to the learned blue/background colors → **BACKGROUND**.

5. GrabCut Does NOT Only Look at Color

This is very important. GrabCut also considers relationships between nearby pixels. Object pixels that are connected and have similar appearance are encouraged to stay together. This helps produce a smoother boundary.

Conceptually:

■ ■ ■ ■
■ ■ ■ ■
■ ■ ■ ■
■ ■ ■ ■

becomes a mask like:

■ ■ ■ ■
■ ■ ■ ■
■ ■ ■ ■
■ ■ ■ ■

6. GrabCut Improves Its First Guess

The first classification may not be perfect, so GrabCut repeats the process.

FIRST GUESS
↓
Learn foreground/background
↓
Classify pixels
↓
Improve boundary
↓
Learn again
↓
Classify again
↓
Better result

In `cv2.grabCut(..., 5, ...)`, the **5** tells GrabCut to run the optimization for 5 iterations.

7. What Is the Mask Doing?

GrabCut creates a mask with one label for every pixel:

Label	Meaning
0 = GC_BGD	Definitely background

1 = GC_FGD	Definitely foreground
2 = GC_PR_BGD	Probably background
3 = GC_PR_FGD	Probably foreground

For example, a final mask might look like:

```
0 0 0 0 0
0 0 1 1 0
0 1 1 1 0
0 0 1 1 0
0 0 0 0 0
```

Here, 0 means remove and 1 means keep. In a real GrabCut result, 2 and 3 can also appear.

8. Understanding the mask2 Code

The code commonly used after GrabCut is:

```
mask2 = np.where(
    (mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD),
    1, 0
).astype('uint8')
```

`mask == cv2.GC_FGD` asks: **Is this pixel definitely foreground?**

`mask == cv2.GC_PR_FGD` asks: **Is this pixel probably foreground?**

`|` means **OR**. Therefore:

Definitely foreground **OR** probably foreground → **KEEP (1)**
Background or probably background → **REMOVE (0)**

9. Finally, the Background Is Removed

Suppose the mask is:

```
0 0 0 0 0
0 0 1 1 0
0 1 1 1 0
0 0 1 1 0
0 0 0 0 0
```

OpenCV essentially performs:

```
IMAGE × MASK
```

Background: Blue × 0 = 0 → removed
Object: Red × 1 = Red → kept

So the result is an image where the object remains and the background is suppressed.

10. Connect Everything Directly to the Code

```
img = cv2.imread("image.jpg")
```

Meaning: Load the image.

```
mask = np.zeros(img.shape[:2], np.uint8)
```

Meaning: Create an empty mask with one position for every image pixel.

```
rect = (10, 10, 300, 400)
```

Meaning: Tell GrabCut where the object is likely to be.

```
bgdModel = np.zeros((1,65), np.float64)
fgdModel = np.zeros((1,65), np.float64)
```

Meaning: Create storage for the models that GrabCut uses to learn foreground and background.

```
cv2.grabCut(...)
```

Meaning: Start the segmentation/learning process.

```
result = img * mask2[:, :, np.newaxis]
```

Meaning: Keep foreground pixels and suppress background pixels.

11. Complete Working Flow

IMAGE
↓
Draw rectangle around object
↓
Outside rectangle = definite background
Inside rectangle = uncertain
↓
Initial foreground/background guess
↓
Learn color distributions
↓
Classify pixels
↓
Consider neighboring pixels and boundaries
↓
Improve segmentation
↓
Repeat for several iterations
↓
Final mask
↓
Keep foreground + suppress background

12. Complete Simple Code

```
import cv2
import numpy as np

# Load image
img = cv2.imread("image.jpg")

# Create empty mask
mask = np.zeros(img.shape[:2], np.uint8)

# Rectangle around the object
rect = (10, 10, img.shape[1]-20, img.shape[0]-20)

# Models used by GrabCut
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)

# Run GrabCut
cv2.grabCut(
img, mask, rect,
bgdModel, fgdModel,
5, cv2.GC_INIT_WITH_RECT
)

# Keep definite + probable foreground
mask2 = np.where(
```

```
(mask == cv2.GC_FGD) |
(mask == cv2.GC_PR_FGD),
1, 0
).astype('uint8')

# Remove background
result = img * mask2[:, :, np.newaxis]

cv2.imshow("Original", img)
cv2.imshow("Foreground", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

■ One-Line Concept

GrabCut starts with a rectangle, learns the appearance of foreground and background, classifies pixels using those models plus relationships between nearby pixels, repeatedly improves the boundary, and produces a mask that separates the object from the background.

Easy Memory Trick

Rectangle → Learn → Classify → Check neighbors → Repeat → Mask → Remove background